

DAY 6 – Java Application Minikube deployment

Minikube deployment of java:

Minikube Command:

- Minikube start
- Minikube status
- Sudo nano deploy.yaml
- Sudo nano pod.yaml
- Kubectl apply -f pod.yaml
- Kubectl apply -f deploy.yaml
- Minikube service my-service
- For permission to, sudo visudo
- Kubectl get node
- Configuring config file in ~/.kube, Sudo vi config

Sudo visudo: update this, jenkins ALL=(ALL) NOPASSWD: ALL

Data updation in config file:

- sudo cat /home/nidharsan/.minikube/ca.crt
- sudo cat /home/nidharsan/.minikube/ca.crt | base64 -w 0; echo
- sudo cat /home/nidharsan/.minikube/profiles/minikube/client.crt | base64 -w 0; echo
- sudo cat /home/nidharsan/.minikube/profiles/minikube/client.key | base64 -w 0; echo

Deployment.yaml:

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-deploy

labels:

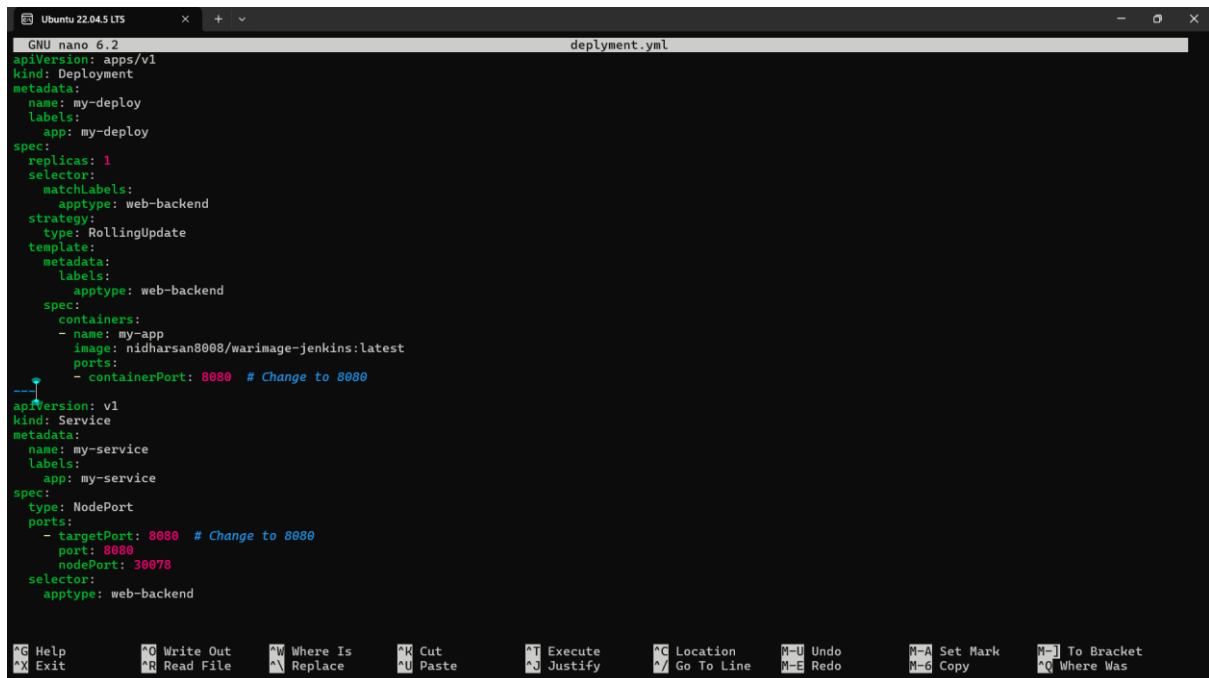
```
    app: my-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      apptype: web-backend
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        apptype: web-backend
    spec:
      containers:
        - name: my-app
          image: nidharsan8008/warimage-jenkins1:latest
          ports:
            - containerPort: 8080 # Change to 8080
---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  labels:
    app: my-service
spec:
  type: NodePort
  ports:
    - targetPort: 8080 # Change to 8080
```

port: 8080

nodePort: 30078

selector:

apptype: web-backend



The screenshot shows a terminal window titled 'Ubuntu 22.04.5 LTS' with a nano editor editing a file named 'deployment.yml'. The manifest defines two Kubernetes resources: a Deployment named 'my-deploy' and a Service named 'my-service'. The Deployment has one replica of 'my-app' using the 'nldharsan8008/warimage-jenkins:latest' image, with container port 8080. The Service is of type 'NodePort', exposing port 8080 on the node (30078) to the container (8080). The selector for both is 'apptype: web-backend'. The nano editor's status bar at the bottom shows various keyboard shortcuts like ^G Help, ^O Write Out, ^W Where Is, etc.

```
GNU nano 6.2 deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deploy
  labels:
    app: my-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      apptype: web-backend
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        apptype: web-backend
    spec:
      containers:
        - name: my-app
          image: nldharsan8008/warimage-jenkins:latest
          ports:
            - containerPort: 8080 # Change to 8080
---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  labels:
    app: my-service
spec:
  type: NodePort
  ports:
    - targetPort: 8080 # Change to 8080
      port: 8080
      nodePort: 30078
  selector:
    apptype: web-backend
```

JenkinsFile :

pipeline {

agent any

stages {

stage('Clone Git Repository') {

steps {

git 'https://github.com/NIDHARSAN-V/Devops_Guvi_WAR.git'

}

}

stage('Build WAR File') {

```

    steps {
        sh 'mvn clean package' // Build WAR file using Maven
    }
}

stage('Build Docker Image') {
    steps {
        script {
            sh 'docker build -t warimage-jenkins .'
            sh 'docker tag warimage-jenkins nidharsan8008/warimage-jenkins1'
        }
    }
}

// stage('Run Docker Container') { // Optional stage to test the container
//  steps {
//    script {
//        sh 'docker run -d -p 8888:8080 --name war-container-jenkins warimage-
jenkins'
//    }
// }
// }

stage('Push to Docker Hub') {
    steps {
        script {
            withDockerRegistry(credentialsId: 'dockerhub_token', url:
'https://index.docker.io/v1/') {
                sh 'docker push nidharsan8008/warimage-jenkins1'
            }
        }
    }
}

```

```
    }  
  }  
}
```

```
stage('Deploy with deployment.yml') {  
  steps {  
    withKubeConfig(caCertificate: '', clusterName: 'minikube ', contextName:  
'minikube', credentialsId: 'MiniKube_ID', namespace: '', restrictKubeConfigAccess: false,  
serverUrl: 'https://192.168.49.2:8443') {  
      sh 'kubectl apply -f deployment.yml'  
    }  
  }  
}
```

```
// stage('Test') {  
  // steps {  
    //   withKubeConfig(caCertificate: '', clusterName: 'minikube ', contextName:  
'minikube', credentialsId: 'MiniKube_ID', namespace: '', restrictKubeConfigAccess: false,  
serverUrl: 'https://192.168.49.2:32274') {  
      //     sh "  
      //   }  
      // }  
    // }  
  }
```

```
  }  
}
```

Dashboard > War_with_JYD > Configuration

Configure

- General
- Triggers**
- Pipeline
- Advanced

☐ GitHub hook trigger for GIT/SCM polling ?

☐ Poll SCM ?

☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

```
Script ?
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
stage('Deploy with deployment.yml') {
    steps {
        withKubeConfig(caCertificate: '', clusterName: 'minikube', contextName: 'minikube', credentialsId: 'Minikube_ID', namespace: 'default') {
            sh 'kubectl apply -f deployment.yml'
        }
    }
}

// stage('Test') {
```

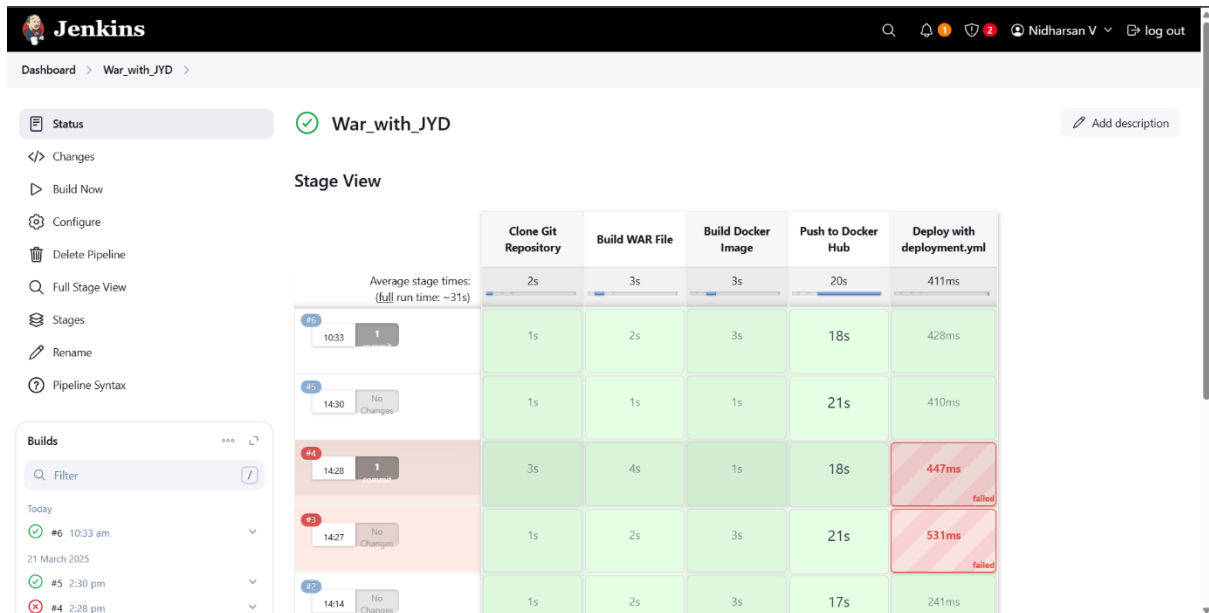
☒ Use Groovy Sandbox ?

Save Apply

Dashboard > War_with_JYD > #6

```
f844dcf94898: Layer already exists
39cf0ac89a5a: Layer already exists
4b7c01ed0534: Layer already exists
a2360fa54a40: Pushed
latest: digest: sha256:4d0066d149b2042127277919c0b527948bd208bb691cccbef7e1af9a528bc4 size: 2409
[Pipeline] }
[Pipeline] // withDockerRegistry
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy with deployment.yml)
[Pipeline] withKubeConfig
[Pipeline] {
[Pipeline] sh
+ kubectl apply -f deployment.yml
deployment.apps/my-deploy configured
service/my-service unchanged
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeConfig
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Stage View of pipeline project:



- Minikube srv my-service

War_with_JYD #6

Dashboard [jenkins]

127.0.0.1:8909/my-app

127.0.0.1:8909/my-app/

Hello World!

Ubuntu 22.04.5 LTS

```

name: context_info
namespace: default
user: minikube
name: minikube
current-context: minikube
kind: Config
preferences: {}
users:
- name: minikube
  user:
    client-certificate: /home/ubuntu/.minikube/profiles/minikube/client.crt
    client-key: /home/ubuntu/.minikube/profiles/minikube/client.key
visudo: /etc/sudoers.tmp unchanged
minikube kube sudo visudo
minikube kube kubect get services
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
kubernetes ClusterIP  10.96.0.1        <none>            443/TCP      24h
my-service NodePort  10.104.97.112   <none>            8080:30078/TCP 20h
minikube kube minikube service my-service
NAMESPACE   NAME      TARGET PORT      URL
default     my-service  8080             http://192.168.49.2:30078

Starting tunnel for service my-service.
NAMESPACE   NAME      TARGET PORT      URL
default     my-service  8080             http://127.0.0.1:45935

Opening service default/my-service in default browser...
http://127.0.0.1:45935
! Because you are using a Docker driver on linux, the terminal needs to be
open to run it.

ubuntu ▶ nidharsan ▶ 10:32 ▶ .kube°C ▶ Stopped tunnel for service my-
service.
minikube kube kubect port-forward svc/my-service 8909:8080
Forwarding from 127.0.0.1:8909 -> 8080
Forwarding from [::]:8909 -> 8080
Handling connection for 8909
Handling connection for 8909

```

